DISTRIBUTED PATTERNS

Транзакція закінчується на межі процесу. Що робити з проміжком, у який вміщається падіння: outbox, CDC, saga, ідемпотентність

Заняття №22

Node.js Pro

Лектор
Кирило Орлов

r_d

СТЕК НА СЬОГОДНІ: POSTGRES · БРОКЕР · RELAY · КОНСЮМЕР

Перша половина — втрата	Друга половина — дублі
dual write · два записи у дві системи	ідемпотентність · дубль не болить
outbox · два записи зводяться до одного	exactly-once · чесна відповідь
relay · SELECT … FOR UPDATE SKIP LOCKED	saga · retry · circuit breaker · DLQ
CDC · журнал бази замість опитування	спершу втрата, потім дублі, потім сусід

НАСКРІЗНА НИТКА

Один домен Marketplace (замовлення → резерв складу → сповіщення) проживає весь шлях: dual write → outbox → relay → консюмер із дедупом → saga. Кожне число на слайдах — реальний вивід із репо цієї лекції, і на ефірі ми його запустимо.

ПЛАН ЗАНЯТТЯ



- Dual write: два записи у дві системи — і проміжок між ними
- Transactional outbox: один COMMIT замість двох записів
- Relay: SELECT … FOR UPDATE SKIP LOCKED і чотири граблі
- CDC і скар про replication slot · вебхук провайдера: підпис і дедуп
- Ідемпотентність: три рівні, ефект застосовується один раз
- Exactly-once чесно: доставки не існує, результат — існує
- Saga · retry з джитером · circuit breaker · DLQ · ДЗ #22

СЬОГОДНІ ПОБАЧИТЕ НА ВЛАСНІ ОЧІ

01. dual write

SIGKILL між COMMIT і publish: замовлення є, у черзі брокера 0

02. дзеркало

publish пройшов, транзакцію відкотили: у черзі подія-сирота

03. relay

три релеї без локів дають 60 дублів, зі SKIP LOCKED — нуль

04. at-least-once

релей убито після publish: 10 доставок на 5 подій, унікальних 5

05. дедуп

наївний консюмер: склад 100 → 90; природний ключ: 100 → 95

06. усе разом

12 доставок, 8 ефектів, склад 100 → 92: результат один раз

НЕМАЄ ОПЕРАЦІЇ «ЗРОБИТИ»



НАСКРІЗНА ТЕЗА

У розподіленій системі немає операції «зробити».
Є «попросити і не дізнатися результату».
Невідомість результату — НОРМАЛЬНИЙ режим, не крайній випадок.

невідомий результат → повтор
безпечний повтор → ідемпотентний ефект
немає спільного COMMIT → outbox сусід лежить → breaker

ЗА МЕЖЕЮ: консенсус (Raft/Paxos) · Redlock і локи — L#23 · service-mesh retry — L#28 · механіка брокерів — L#19-20 · платіж — приклад, а не тема: беремо намір, ідемпотентність, вебхук; регуляторика, PCI і схеми підпису — L#27

CAP ТОЧНО · PACELC · РІВЕНЬ ОПЕРАЦІЇ

CAP

за розділу мережі не можна мати лінеаризованість і доступність разом — і вибір є ЛИШЕ під час розділу

PACELC

if P then A/C, Else L/C — платиш і без розділу: або латентністю, або узгодженістю

операція

резерв останнього товару вимагає лінеаризованості, лічильник переглядів — ні: одна система, два рішення

Пастка співбесіди
Не кажіть «двоє з трьох» — Brewer сам через 12 років назвав це формулювання оманливим.

DUAL WRITE: ДВІ ДІРКИ НАЖИВО

два записи у дві системи: спершу рядок у Postgres, потім publish

процес помер між ними: замовлення є, події немає — склад не резервує

дзеркало: publish пройшов, транзакцію відкотили — подія-сирота

два контейнери — два менеджери ресурсів: спільного COMMIT немає

Помилка publish НЕ означає, що повідомлення не доставлено — загубитись могла відповідь брокера. Тому «rollback у catch» не рятує.



```
крок 1 · dual write

orders.save(dto) (1)
broker.publish() (2)

1.1 SIGKILL між (1)
    і (2):
 замовлення #1: є
 у черзі брокера: 0
 → подія загублена

1.2 publish пройшов,
    ми відкотили tx:
 замовлення: НЕМАЄ
 у черзі брокера: 1
 → подія-сирота
```

«ШУХЛЯДА ВИХІДНИХ» — АНКОР ЛЕКЦІЇ

КОНЦЕПТ

- orders + outbox, один COMMIT
- relay або CDC
- at-least-once
- ідемпотентний консюмер
- dead letter

АНАЛОГІЯ (ШУХЛЯДА)

- договір і конверт в одній шухляді
- кур’єр, який забирає конверт
- кур’єр не пам’ятає, чи доніс
- список номерів отриманих конвертів
- коробка з поверненим

Де аналогія ламається: кур’єр не гарантує ПОРЯДОК — його забезпечують окремо, ключем партиціонування. Віднести конверт двічі кур’єр може; віднести те, чого в шухляді не було, — не може.



TRANSACTIONAL OUTBOX: ОДИН COMMIT

```
-- поля за зразком Debezium Outbox Event Router SMT + поля під relay
CREATE TABLE outbox (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type text NOT NULL,   -- 'Order' → топік
  aggregate_id   text NOT NULL,   -- '42'    → ключ партиціонування
  type           text NOT NULL,   -- 'OrderPlaced'
  payload        jsonb NOT NULL,  -- СВІДОМО спроектований контракт
  created_at     timestamptz NOT NULL DEFAULT now(),
  published_at   timestamptz,     -- NULL = ще не винесено
  attempts       int NOT NULL DEFAULT 0
);

-- «хвіст» невинесених лишається малим, хоч таблиця й росте
CREATE INDEX outbox_unpublished_idx
  ON outbox (created_at) WHERE published_at IS NULL;

-- запис: ОДНА транзакція, два INSERT-и
BEGIN;
  INSERT INTO orders (id, status) VALUES ($1, 'PENDING');
  INSERT INTO outbox (aggregate_type, aggregate_id, type, payload)
    VALUES ('Order', $1, 'OrderPlaced', $2);
COMMIT;   -- обидва рядки або жоден
```

Той самий SIGKILL у ту саму точку — інший результат

1 · INSERT orders + INSERT outbox — в одній транзакції

2 · COMMIT — обидва рядки або жоден: два записи звелись до одного

3 · у брокері 0 — і це НОРМАЛЬНО: не «загубили», а «ще не віддали»

aggregate_id — ключ партиціонування; payload — свідомий контракт

частковий індекс WHERE published_at IS NULL — той самий, що на L#12

```
-- крок 3 · relay: той самий SKIP LOCKED, що на L#14, — але над
--          таблицею вихідних подій (03-relay-skip-locked.ts:53)
SELECT * FROM outbox
 WHERE published_at IS NULL
 ORDER BY created_at, id
   FOR UPDATE SKIP LOCKED
 LIMIT 100;

-- три релеї над однією таблицею, 30 подій, пачка по 5:
--   [naive      ] у брокері 90 повідомлень: унікальних 30, ДУБЛІВ 60
--   [for-update ] 30 повідомлень: ДУБЛІВ 0 — і нуль паралельності
--   [skip-locked] 30 повідомлень: ДУБЛІВ 0, час ≈ ідеальному
--
-- СТАБІЛЬНЕ ТУТ: нуль дублів і форма часу. Розподіл рядків між
-- релеями у FOR UPDATE щоразу інший — це черга на локу, не баг.

-- далі, окремою дією: publish → UPDATE outbox SET published_at
--   ↑ між цими двома діями релей може померти.
```



ЧОТИРИ ГРАБЛІ OUTBOX

1 · published_at ставиться ПІСЛЯ publish — і це не атомарно

2 · ORDER BY — це порядок вибірки, а не порядок доставки

3 · затримка polling → LISTEN/NOTIFY (приходить лише після COMMIT)

4 · таблиця росте і гальмує власний індекс: DELETE одразу, фонова чистка або партиціонування

обирають між ДУБЛЕМ і ВТРАТОЮ: дубль лікується, втрата — ні

анти-патерн: ORM-сутність у payload = схема БД як публічний API

CDC: ЖУРНАЛ БАЗИ ЗАМІСТЬ ОПИТУВАННЯ

01. polling: свій воркер, латентність = інтервал опитування

02. CDC: реальний час; ціна — Kafka Connect і слот у проді

03. replication slot утримує WAL, доки консюмер не підтвердив прочитане

04. один-два сервіси без Kafka — polling; багато сервісів і Kafka вже є — CDC

05. Outbox Event Router SMT робить із рядка outbox повідомлення

СКАР: СЛОТ, ЯКИЙ ЗАПОВНИВ ДИСК

max_slot_wal_keep_size за замовчуванням = -1 — утримання WAL НЕОБМЕЖЕНЕ. Конектор ліг у п’ятницю, у понеділок pg_wal заповнив диск, і база СТАЛА

ліки: моніторити restart_lsn, виставити ліміт, у PG18 — idle_replication_slot_timeout. Ціна запобіжника — інвалідація слоту: обираєш між «впаде база» і «впаде конектор»

⚠ анкор: WAL і CDC — це СТЕНОГРАМА засідання, усе дослівно. Outbox — ПРОТОКОЛ: те, що ви свідомо вирішили опублікувати. Найкраща збірка — outbox для сенсу, CDC для доставки. І це не event sourcing



ВЕБХУК ПРОВАЙДЕРА: ТОЙ САМИЙ ЖУРНАЛ З ІНШОГО БОКУ

```
ДЕМО E · webhook-verify-demo.mjs · без Docker і мережі

1. сирі байти як прийшли          : true
2. сума 90000 -> 900000           : false
3. розпарсив і зібрав назад        : false
   довжина сирого / перезібраного : 99 / 82

4. перша доставка evt_7Kq         : ефект застосовано
   друга доставка evt_7Kq         : дубль, ефекту немає
   третя доставка evt_7Kq         : дубль, ефекту немає

сире тіло → підпис → дедуп за id події → звірка суми
Той самий журнал з іншого боку: CDC — ви читаєте чужий
журнал, вебхук — чужа система дзвонить вам.
```



AT-LEAST-ONCE НАЖИВО: 10 ДОСТАВОК НА 5 ПОДІЙ

```
крок 4 · SIGKILL між publish і UPDATE published_at

[kill]   брокер підтвердив 5 publish → SIGKILL
[брокер] у черзі: 5
[outbox] published_at IS NULL: 5 — транзакцію релея обірвало
[relay]  винесено рядків: 5     ← прохід 2, після рестарту

✗ у брокері всього повідомлень: 10 на 5 подій
✗ унікальних messageId: 5, з них доставлено двічі: 5
```

Дубль зробив НАШ РЕЛЕЙ, а не брокер: жодна галочка в RabbitMQ чи Kafka його не прибере. Поміняти publish і UPDATE місцями — отримати ВТРАТУ замість дубля. Дубль лікується дедупом, втрата — нічим.



ІДЕМПОТЕНТНІСТЬ — ВЛАСТИВІСТЬ ЕФЕКТУ

ідемпотентність — властивість ЕФЕКТУ, а не відповіді

повторний DELETE може віддати 404 і лишатись ідемпотентним

рівень 1 — природний ключ: ON CONFLICT (order_id, sku) DO NOTHING

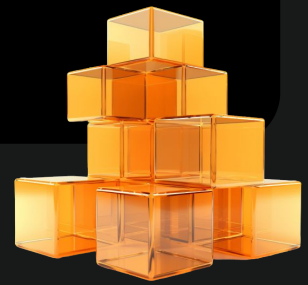
рівень 2 — inbox: позначка й ефект в ОДНІЙ транзакції

позначка в Redis + ефект у Postgres = знову dual write

ідемпотентність операції ≠ ключ ідемпотентності: ключ — лише механізм

```
крок 5 · дедуп на консюмері

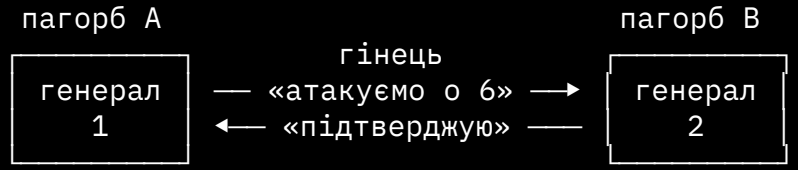
5.1 наївний UPDATE:
    склад 100 → 90
    (мало бути 95)
5.2 природний ключ:
    склад 100 → 95
5.3 inbox:
    оброблено 5,
    відкинуто дублем 5
5.4 Set у пам'яті, 2 поди:
    push 10 (мало бути 5)
```



ТРИ РІВНІ ІДЕМПОТЕНТНОСТІ

Рівень	Механіка	Що показало демо
1 · природний ключ	ON CONFLICT (order_id, sku) DO NOTHING	склад 100 → 95
2 · inbox	processed_messages + ефект в ОДНІЙ транзакції	оброблено 5, відкинуто 5
3 · API-край	Idempotency-Key — це МЕХАНІЗМ, а не сама властивість	дубль запиту не створить друге замовлення
✕ анти-патерн	дедуп у Set у пам’яті процесу	2 поди → push 10 замість 5
✕ анти-патерн 2	позначка в Redis + ефект у Postgres	це знову dual write

ДВОЄ ГЕНЕРАЛІВ — чому exactly-once доставки не існує



ворог у долині: гінця можуть перехопити

Перший не знає, чи дійшов наказ.
Другий не знає, чи дійшло підтвердження.
Третій гонець із підтвердженням підтвердження нічого не міняє:
ЖОДНА скінченна кількість підтверджень не дає впевненості.

АЛЕ атаку можна синхронізувати. Правило:
«атакуємо о шостій, якщо отримав ХОЧА Б ОДИН наказ».

Гінців може прийти багато. Атака буде ОДНА.



ЧОТИРИ СЕМАНТИКИ ДОСТАВКИ

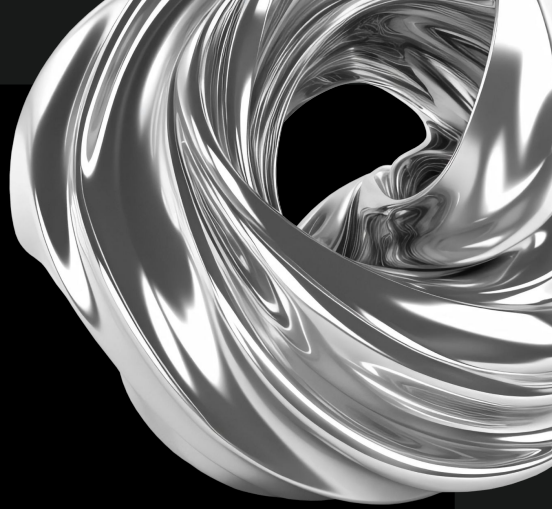
Термін	Що означає	Досяжно
at-most-once delivery	може загубитись, дубля не буде	✅ тривіально: ack до обробки
at-least-once delivery	не загубиться, може прийти кілька разів	✅ дефолт серйозних брокерів
exactly-once delivery	доставлено рівно один раз	✕ НЕ ІСНУЄ з ненадійною мережею
exactly-once effect	ефекти застосовані рівно один раз	✅ = at-least-once + ідемпотентність
EOS у Kafka	read-process-write усередині Kafka	✅ лише всередині Kafka

```
// KAFKA · документація, розділ Message Delivery Semantics
   «Many systems claim to provide "exactly-once" delivery semantics,
    but it is important to read the fine print, because sometimes
    these claims are misleading.»

   «Exactly-once delivery for other destination systems generally
    requires cooperation with such systems… Otherwise, Kafka
    guarantees at-least-once delivery by default.»

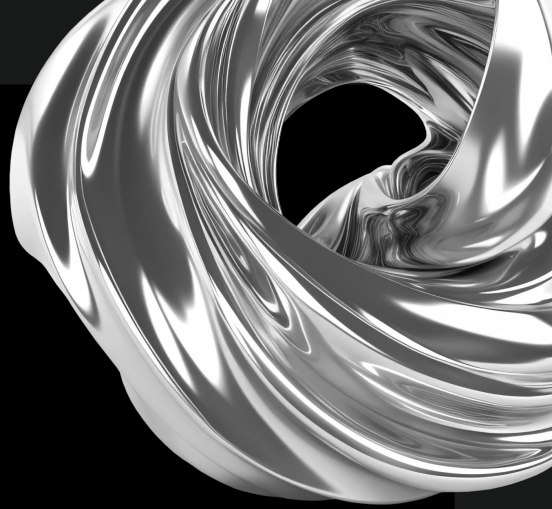
// DEBEZIUM · сторінка Exactly-once delivery
   «Debezium provides at-least-once delivery guarantees. This means
    that no changes are missed; however, in some cases, a record
    may be delivered more than once.»

// ЗВІРЕНІ ДЕФОЛТИ KAFKA 4.3
   acks = all · enable.idempotence = true · transactional.id = null
   isolation.level = read_uncommitted  ← консюмер за замовчуванням
                                         читає й аборчені записи
```



```
// SAGA — послідовність ЛОКАЛЬНИХ транзакцій + компенсації
T1  orders     створити замовлення PENDING
T2  inventory  зарезервувати товар    → C2: зняти резерв
T3  shipping   створити відправлення  → C3: скасувати відправлення
T4  orders     перевести в CONFIRMED

// 1 · компенсація ≠ rollback. Rollback стирає слід; компенсація —
//     нова бізнес-дія: не «видалити лист», а надіслати другий:
//     «замовлення скасовано»
// 2 · немає ізоляції: saga дає ACD без I — проміжний стан видно
//     іншим. Протиотрута — СЕМАНТИЧНИЙ ЛОК: status = PENDING,
//     який інші поважають
// 3 · pivot: некомпенсовні кроки — якомога пізніше. До pivot усе
//     компенсовно, після неї сага МУСИТЬ дійти вперед
//
// Термін: Garcia-Molina & Salem, 1987. Оркестратор — наступний слайд →
// Анти-патерн: синхронний ланцюжок
// HTTP у try/catch, названий сагою: три сервіси по 99.9% = 99.7%
```



```
// ТА САМА САГА — КОДОМ · Temporal, src/workflows.ts
export async function orderWorkflow(input: OrderInput): Promise<string> {
  const id = input.orderId;
  const compensations: Array<() => Promise<void>> = [];
  try {
    await a.reserveStock(id);
    compensations.unshift(() => a.releaseStock(id));
    await a.chargePayment(id);
    compensations.unshift(() => a.refundPayment(id));
    // … 37–43: таймер, total, slowShip — на #33
    await a.shipOrder(id, input.failShip ?? false); // 44, спрощено
    return `SHIPPED ${id} total=${total}`;
  } catch (err) {
    for (const undo of compensations) await undo(); // зворотний порядок
    return `COMPENSATED ${id}: ${(err as Error).message}`;
  }
}

// ./run-demo.sh saga · A-200, failShip: true · dev-ci, 06.10.2026
  [activity] refundPayment(A-200) спроба=1
  [activity] releaseStock(A-200) спроба=1
result=COMPENSATED A-200: Activity task failed
// причина — у err.cause: «склад відмовив: товар пошкоджено»
status=COMPLETED
// Temporal · журнал тримає сервер · ламаємо воркер — #33
```

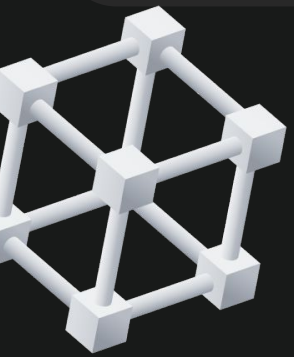
RETRY · BACKOFF · JITTER

без джитера: 24/40 успіхів і 72 відмови «перевантажений» — це задано кодом, а не випадковістю

з джитером «перевантажений» падає практично до нуля, а успіхи — майже до всіх сорока; ці числа ВИПАДКОВІ, читаємо їх з екрана прогону

ретрай того, що не можна ретраїти: 40 × retry = 240 запитів, 200 із них марні; з класифікацією — рівно 40

amplification: три рівні по три спроби = 27 запитів на дні. Retry budget ~10% трафіку (SRE Book). І головне: джитер лікує КОРЕЛЬОВАНІСТЬ, а не брак потужності — при стелі 40 різниця зникає



СТІЙКІСТЬ: ЩО РОБИТЬ КОЖЕН ПАТЕРН

ПАТЕРН	ЩО САМЕ РОБИТЬ
порядок обгортання	timeout → retry → circuit breaker
circuit breaker	три стани; half-open = «спробуємо ОДНЕ»
граблі брейкера	поріг без volumeThreshold
bulkhead	окремий пул; без стелі черги — OOM
dead letter queue	DLQ без алерту і runbook-у = смітник

ПІДСУМКИ · ДЗ #22 · МІСТ ДО L#23

OUTBOX

два записи зводяться до одного COMMIT; ціна закриття втрати — дублі, і це дизайн

СТІЙКІСТЬ

timeout → retry з джитером → circuit breaker → DLQ з алертом і runbook-ом

ДАЛІ

L#23 — Redis: розподілений лок НЕ замінює ідемпотентність

ДЗ #22

7 балів (PROJECT MEDIUM) · дедлайн 22.10 · outbox в одній транзакції + relay зі SKIP LOCKED + дедуп на консюмері. Показати подію ПІСЛЯ фейлу запису

EXACTLY-ONCE

exactly-once ефект = at-least-once доставка + ідемпотентність ефекту

Q&A

???



YOUR PLACE
TO GROW